Lab 2 Part 2 報告：Qwen3-4B LLM Compression

學號：315605019　　日期：2026-11-08

實作設定

四個 case 都有達到要求，下表是 `bash run.sh` 在一張 T4 上的輸出。

Case	方法	Compression Ratio	PPL
Base	FP16	0%	13.64
1	GPTQ INT4 g128 + gate_proj INT3 g128	75.58%	14.02
2	GPTQ INT3 g512	81.02%	15.49
3	Whitened SVD (MLP) + INT8	50.01%（SVD 20.88%）	20.80
4	結構化剪枝 MLP channel + GPTQ INT3 g512	85.23%（剪枝 22.20%）	18.33

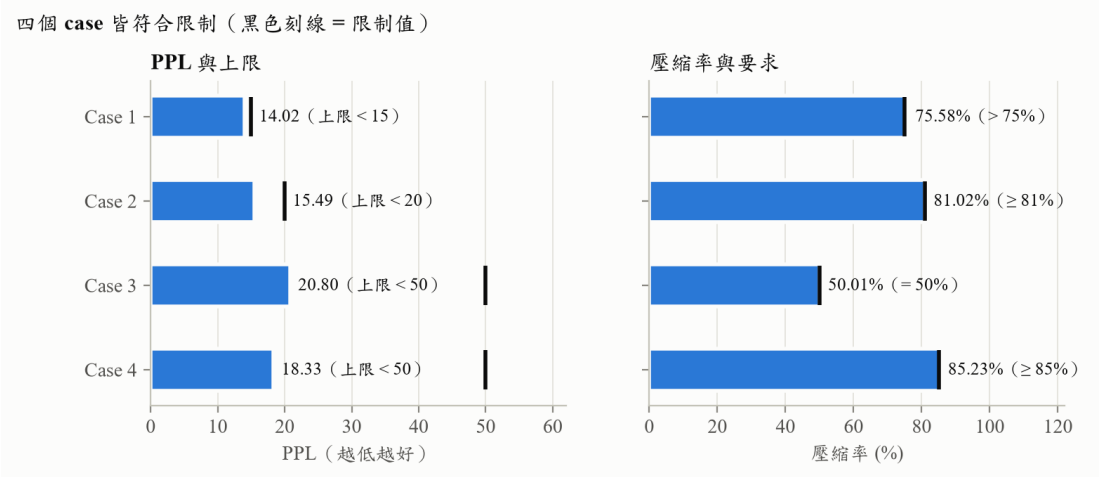


圖 1：四個 case 的 PPL 與壓縮率，黑色刻線為限制值

壓縮對象是 36 層的 q/k/v/o/gate/up/down_proj，共 3.633B 個參數，其中 MLP 的三個 projection 佔 74%。lm_head 和 embedding 共用權重，不列入計算。校正資料取 WikiText-2 train 的 128 段、每段 2048 token，沒有用到 test。所有方法都是 post-training，沒有 fine-tuning。

程式分成 `common.py`（共用的量化、GPTQ、SVD 與 bits 計算）和 `case1.py` 到 `case4.py`（各 case 的設定）。`run_eval.py` 只在載入模型後加一行呼叫 `caseN.compress()`，`evaluate.py` 只補上 `compression_rate()`，PPL 的計算沒有改動。報告中的程式片段為了好讀有稍微簡化，完整版本見程式檔。

共用實作

1. 壓縮率計算

量化採 fake quantization：權重量化後再反量化回 fp16 存回模型，所以壓縮率不能看記憶體，而是依「實際要存多少 bits」計算。每個被壓縮的模組都會記下自己的 `_compressed_bits`。

```
def quant_storage_bits(out_features, in_features, wbits,
                       group_size=-1, sym=False, scale_bits=16):
    gs = in_features if group_size <= 0 else group_size
    n_groups = out_features * math.ceil(in_features / gs)
    # 每個 group：fp16 scale + b-bit zero point
    per_group = scale_bits + (0 if sym else wbits)
    return out_features * in_features * wbits + n_groups * per_group

# evaluate.py
original = sum(model._base_linear_numel.values()) * 16   # 壓縮前用 record_base() 記下
compressed = sum(module_bits(model.get_submodule(n))
                 for n in model._base_linear_numel)
return 1 - compressed / original
```

每個 group 除了 b-bit 權重，還要存一個 fp16 scale 和一個 b-bit zero point，所以平均 bits = b + (16 + b) / g。SVD 的層存兩個量化後的低秩矩陣；剪枝後的層矩陣直接變小，用剪枝後的形狀計算，不需要 mask。`test_common.py` 用小矩陣手算驗證過這些公式。

配置	bits / weight	壓縮率
INT4 g128	4.156	74.0%
INT3 g128	3.148	80.3%
INT3 g512	3.037	81.02%
INT3 per-channel	3.007	81.2%
INT8 per-channel	8.006	50.0%

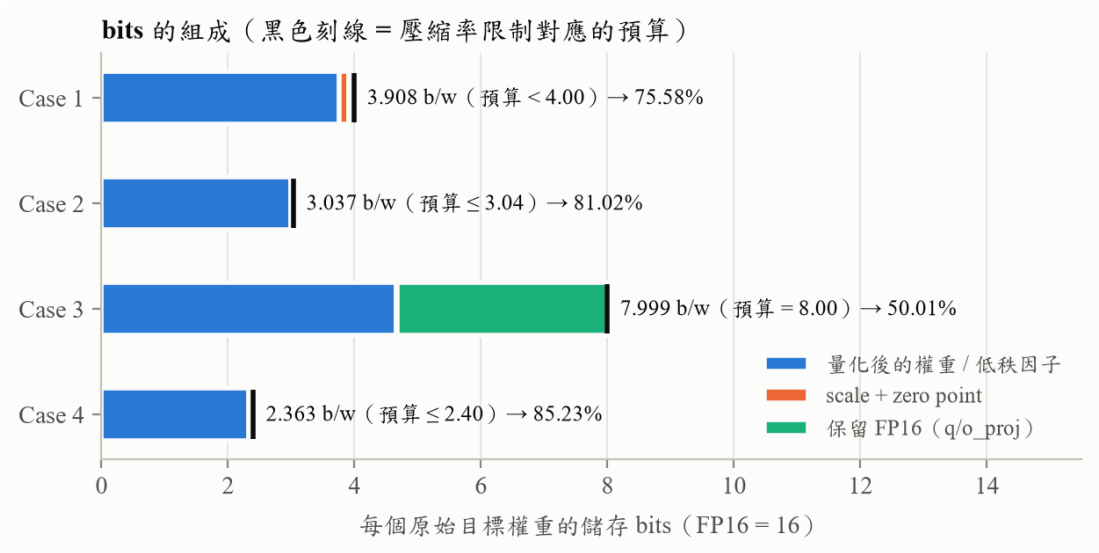


圖 2：各 case 每個原始權重的儲存 bits 組成

2. GPTQ

GPTQ 一次量化一個 column，並把這個 column 的量化誤差，依 Hessian 反矩陣分攤到還沒量化的 column 上，讓整層的輸出誤差最小。我用 PyTorch 自己實作，核心如下：

```
# static groups：先照原本順序算好每個 group 的 scale / zero point
groups = [quant_params(W[:, g:g+gs], wbits, sym, mse) for g in range(0, cols, gs)]
# act-order：依 Hessian 對角線由大到小重排 column
perm = torch.argsort(torch.diag(H), descending=True)
W, H = W[:, perm], H[perm][:, perm]
H += percdamp * torch.mean(torch.diag(H)) * torch.eye(cols)     # dampening
Hinv = torch.linalg.cholesky(torch.cholesky_inverse(torch.linalg.cholesky(H)),
                             upper=True)
# 實際程式以 128 個 column 為一個 block 批次更新
for i in range(cols):
    scale, zero = groups[perm[i] // gs]
    q = quantize(W[:, i], scale, zero, wbits)
    err = (W[:, i] - q) / Hinv[i, i]
    W[:, i:] -= err[:, None] @ Hinv[i, i:][None, :]   # 把誤差補到後面的 column
```

- **act-order**：依 Hessian 對角線由大到小處理 column，先量化對輸出影響大的。
- **static groups**：group 的 scale 和 zero point 在重新排列之前，就照原本的 column 順序算好。這樣 act-order 打亂順序也不需要額外存每個 column 屬於哪個 group。
- **逐層依序**：每一層的校正輸入，是前面已經量化過的層的輸出，所以後面的層能補償前面造成的誤差。

3. MSE clipping

用最大最小值決定量化範圍時，少數極端值會把範圍撐大，讓大部分權重分到的格點很少。所以對每個 group 試著把範圍縮小，挑誤差最小的那一個：

```
for i in range(int(maxshrink * grid)):          # grid = 100, maxshrink = 0.8
    p = 1 - i / grid                            # 範圍縮成 p 倍
    s, z = _range_to_params(p * xmin, p * xmax, maxq, sym)
    err = (quantize(w, s, z, wbits) - w).abs().pow(2.4).sum(dim=1, keepdim=True)
    better = err < best
    best = torch.where(better, err, best)
    scale, zero = torch.where(better, s, scale), torch.where(better, z, zero)
```

這段在 GPTQ 開始前算好每個 group 的 scale 和 zero point，之後 GPTQ 就用這組參數量化。

4. Hessian 收集

GPTQ 和 SVD 都需要每個 Linear 輸入的二階矩 $H = 2XX^T / n$。為了在 T4 上加快速度，我用 fp16 做矩陣乘法，並在所有需要的層都拿到輸入後就中斷 forward，不跑完整層。

```
def _hook(self, module, inp, out):
    x = inp[0].reshape(-1, inp[0].shape[-1])
    ...
    c = x.abs().amax(dim=0).float().clamp(min=1e-6)    # 每個 channel 自己的最大值
    xh = (x.float() / c).half()                        # 縮到 [-1, 1] 再轉 fp16
    self.H += (xh.t() @ xh).float() * (c[:, None] * c[None, :]) * (2 / self.n)
```

一開始我是用整個 batch 的最大值來縮放，結果 Case 3 的 Cholesky 失敗。原因是 Qwen3 的 down_proj 輸入有極端的 outlier channel，最大值和中位數差約 7000 倍，一般的 channel 縮放後太小，在 fp16 下 underflow，H 就不再正定。改成每個 channel 分別縮放後，和 fp64 結果的相對誤差約 1e-4。

Case 1：> 75%，PPL < 15

Compression Strategy

GPTQ 加 MSE clipping，壓縮全部目標層。gate_proj 用 INT3 g128，其餘用 INT4 g128，都是非對稱量化，平均 3.907 bits / weight。

```
INT4 = dict(wbits=4, group_size=128, sym=False, mse=True)
INT3 = dict(wbits=3, group_size=128, sym=False, mse=True)
DEFAULT = INT4
RULES = [(0, 35, "mlp.gate_proj", INT3)]     # 第 0 到 35 層的 gate_proj 改用 INT3
```

選擇理由

壓縮率 > 75% 代表平均要少於 4 bits。INT4 本身加上 scale 和 zero point 就超過 4 bits，所以只用 INT4 一定不夠：

方案	bits / weight	壓縮率	結果
全部 INT4 g128	4.156	74.02%	壓縮率不夠
全部 INT4 per-channel	4.008	74.95%	壓縮率不夠
INT4 g128 + 一部分 INT3 g128	3.907	75.58%	可行

要把平均壓到 4 bits 以下，至少要有 15.5% 的參數改成 INT3。我選擇「把一整種 projection 改成 INT3」，因為設定簡單，而且 gate、up、down 各佔 24.7%，任一種都夠。接著實際測量哪一種對 PPL 影響最小：

改成 INT3 的層	參數佔比	壓縮率	PPL
gate_proj	24.68%	75.58%	**14.02**
down_proj	24.68%	75.58%	14.06
q_proj + o_proj	20.78%	75.33%	14.21
up_proj	24.68%	75.58%	14.31

gate_proj 的 PPL 最低，所以選它。

Result

壓縮率 75.58%，PPL 14.02。

Analysis

MSE clipping 的幫助很明顯（圖 3）。全部 INT4 時，PPL 從 14.16 降到 13.84；加上 gate INT3 後，從 14.29 降到 14.02。最後的結果甚至比沒有 MSE 的全 INT4 還好。

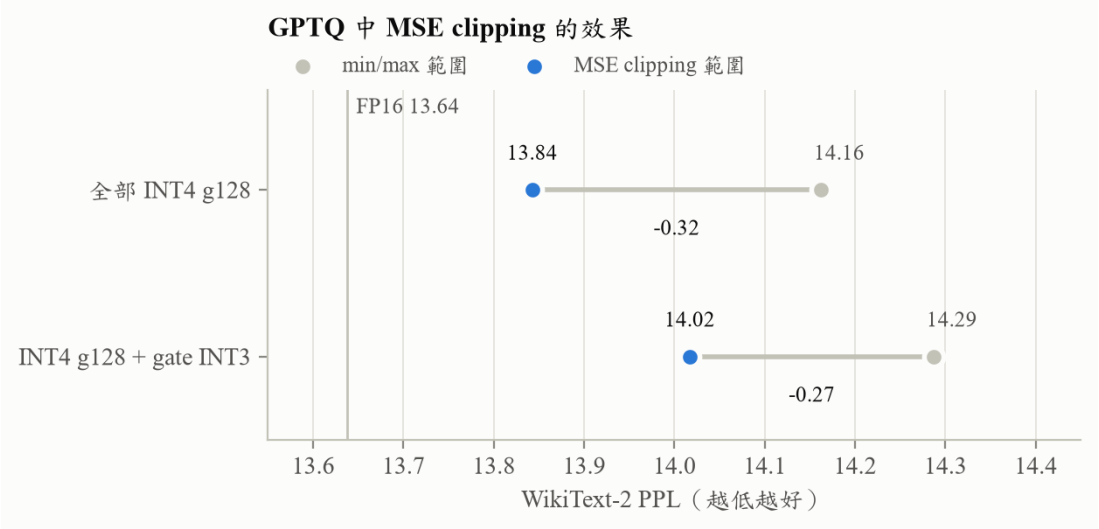


圖 3：GPTQ 中 MSE clipping 的效果

Case 2：≥ 81%，PPL < 20

Compression Strategy

全部目標層使用 GPTQ INT3、group size 512、非對稱量化，加上 MSE clipping，平均 3.037 bits / weight。

```
DEFAULT = dict(wbits=3, group_size=512, sym=False, mse=True)
RULES = []
```

選擇理由

81% 代表平均最多 3.04 bits。3-bit 的權重已經佔掉 3 bits，scale 和 zero point 最多只能再用 0.04 bits，所以重點是選 group 的大小：

方案	bits / weight	壓縮率	PPL	結果
INT3 g128	3.148	80.3%	–	壓縮率不夠
INT3 g256 + 8-bit scale	3.043	80.98%	–	壓縮率差一點
INT3 g512	**3.037**	**81.02%**	**15.49**	通過
INT3 per-channel	3.007	81.21%	22.89	PPL 太高

g512 是能過 81% 的最小 group size。group 越小，scale 越能貼近局部的數值範圍，誤差越小，所以在能過門檻的前提下選最小的。

Result

壓縮率 81.02%，PPL 15.49。

Analysis

per-channel 和 g512 只差 0.03 bits，PPL 卻從 22.89 降到 15.49。可見在 3-bit 時，group 的粒度影響很大，多存一點 scale 非常划算。和 Case 1 相比，平均從 3.907 降到 3.037 bits，PPL 只多了約 1.5。

Case 3：SVD ≥ 20%，Overall = 50%，PPL < 50

Compression Strategy

對 gate/up/down_proj 做 whitened SVD（SVD-LLM 的做法），36 層共用 rank 1455。分解出來的兩個矩陣用 INT8 per-row 量化，k/v_proj 用 INT8，q/o_proj 保留 FP16。

```
def whitened_svd(weight, H, rank, percdamp=0.01):
    damp = percdamp * torch.mean(torch.diag(H))
    while True:                                    # Cholesky 失敗就加大 damping
        S, info = torch.linalg.cholesky_ex(H + damp * eye)
        if info.item() == 0:
            break
        damp *= 10
    U, sig, Vh = torch.linalg.svd(W @ S, full_matrices=False)
    root = sig[:rank].sqrt()
    A = U[:, :rank] * root                                    # [out, rank]
    B = torch.linalg.solve_triangular(S, root[:, None] * Vh[:rank],
                                      upper=False, left=False)    # [rank, in]
    return B, A                                               # W ≈ A @ B
```

H 是輸入的二階矩，S 是它的 Cholesky 因子（$H = SS^T$）。對 W·S 做 SVD 再截斷，丟掉的部分剛好是對輸出 WX 影響最小的方向；最後乘回 S^{-1} 得到 B。rank 用二分搜尋決定：

```
def solve_rank(model, target_rate):
    lo, hi = 1, 2560
    # 找「整體壓縮率仍 >= 50%」的最大 rank
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if 1 - _plan(model, mid)[0] / base_bits >= target_rate:
            lo = mid
        else:
            hi = mid - 1
    return lo
```

`_plan()` 會依照目前的設定算出整個模型的 bits，所以 rank 會是讓整體壓縮率最接近 50% 又不低於 50% 的那一個。

選擇理由

- **為什麼用 whitened SVD**：一般的 SVD 只讓權重誤差最小，但 down_proj 的輸入有極端的 outlier channel，權重誤差小不代表輸出誤差小。whitening 讓截斷時直接以輸出誤差為準。
- **為什麼只分解 MLP**：MLP 佔全部參數的 74%，只分解這三種就能讓 SVD 部分超過 20%，也只需要一個共用的 rank。
- **rank 的上下限**：rank r 的低秩分解要存 r(m + n) 個參數，r 必須小於 mn / (m + n) = 2027 才省空間。另外要讓 SVD 部分 ≥ 20%，r 最多只能到 1479。
- **為什麼其他層用 INT8 和 FP16**：題目要求整體剛好 50%，而低秩截斷是這個 case 主要的誤差來源，所以希望 rank 越大越好。如果 q/o_proj 也用 INT8，整體壓縮率會超過 50%，要拉回 50% 就得把 rank 加到 1479 以上，SVD 部分就會低於 20%。因此

q/o_proj 保留 FP16；它本身沒有誤差，又剛好讓 rank 1455 落在兩個限制之間。其他層用 INT8，幾乎不影響 PPL。

Result

整體壓縮率 50.01%，其中 SVD 部分 20.88%，PPL 20.80。

Analysis

PPL 增加了 7.2，比 Case 1、2 只靠量化時的 0.4 到 1.9 多很多。我推測主要來自低秩截斷，因為 INT8 幾乎不影響 PPL，但沒有單獨量測。在這個模型上，要省下同樣的空間，低秩分解的代價明顯比量化大。

Case 4：剪枝 ≥ 20%，Overall ≥ 85%，PPL < 50

Compression Strategy

每層刪掉 30% 的 MLP intermediate channel，然後對全部目標層做 GPTQ INT3 g512 加 MSE clipping。剪枝和補償的程式如下：

```
def prune_mlp(mlp, H_down, keep_n):
    Wd, H = mlp.down_proj.weight.double(), H_down.double()
    score = (Wd ** 2).sum(dim=0) * torch.diag(H)           # channel j 的輸出能量
    keep = torch.topk(score, keep_n).indices.sort().values
    rm = ...                                                # 其餘要刪掉的 channel
    Hkk = H[keep][:, keep] + damp * I
    rhs = Wd[:, rm] @ H[rm][:, keep]
    Wd_new = Wd[:, keep] + torch.linalg.solve(Hkk, rhs.t()).t()   # 最小平方補償
    mlp.gate_proj = _linear(mlp.gate_proj.weight[keep])    # 刪 row
    mlp.up_proj = _linear(mlp.up_proj.weight[keep])        # 刪 row
    mlp.down_proj = _linear(Wd_new)                         # 刪 column 並補償
```

刪掉一個 intermediate channel，等於同時刪掉 gate_proj 和 up_proj 的一個 row，以及 down_proj 的一個 column。channel 的重要度是 $\|W_down[:, j]\|^2 \cdot H_jj$，也就是它對 MLP 輸出的能量。刪除後用最小平方法重新計算 down_proj，讓保留的 channel 盡量重現原本的輸出：

$$W' = W_keep + W_rm \cdot H_rk \cdot H_kk^{-1}$$

每層的順序是：先量化 attention 的四個 projection，收集 down_proj 的 H 後剪枝 MLP，再量化 gate/up_proj，最後重新收集剪枝後 down_proj 的 H 再量化它。這些都是閉式解，不需要 gradient 訓練。

選擇理由

85% 代表平均只能用 2.4 bits。量化部分沿用 Case 2 的 INT3 g512（3.037 bits），再比較幾種剪枝方式要剪多少才能過門檻：

剪枝方式	額外成本	需要的剪枝量	結果
2:4 稀疏	每個非零值 2-bit index	固定 50%	2.52 bits，只有 84.3%，不夠
非結構化	每個權重 1-bit mask	至少 53.9%	剪太多
結構化（MLP channel）	無	參數至少 21%	可行

結構化剪枝讓矩陣真的變小，沒有額外成本，所以需要剪的量最少。只剪 MLP 是因為它佔 74% 的參數，而且一個 channel 同時縮小三個矩陣。要刪掉 21% 的參數，MLP channel 至少要剪 28.4%，我取 30%，實際剪枝率 22.20%。剩下 77.8% 的參數乘上 3.037 bits 約 2.36 bits，對應 85.2%，和實際結果一致。

Result

剪枝率 22.20%，整體壓縮率 85.23%，PPL 18.33。

Analysis

和 Case 2 相比，同樣是 INT3 g512，多剪掉 22% 的參數，PPL 從 15.49 升到 18.33，離 50 的上限還很遠。我沒有另外測量「不做補償」的版本，所以無法量化最小平方補償單獨帶來多少改善。

總結

Trade-off

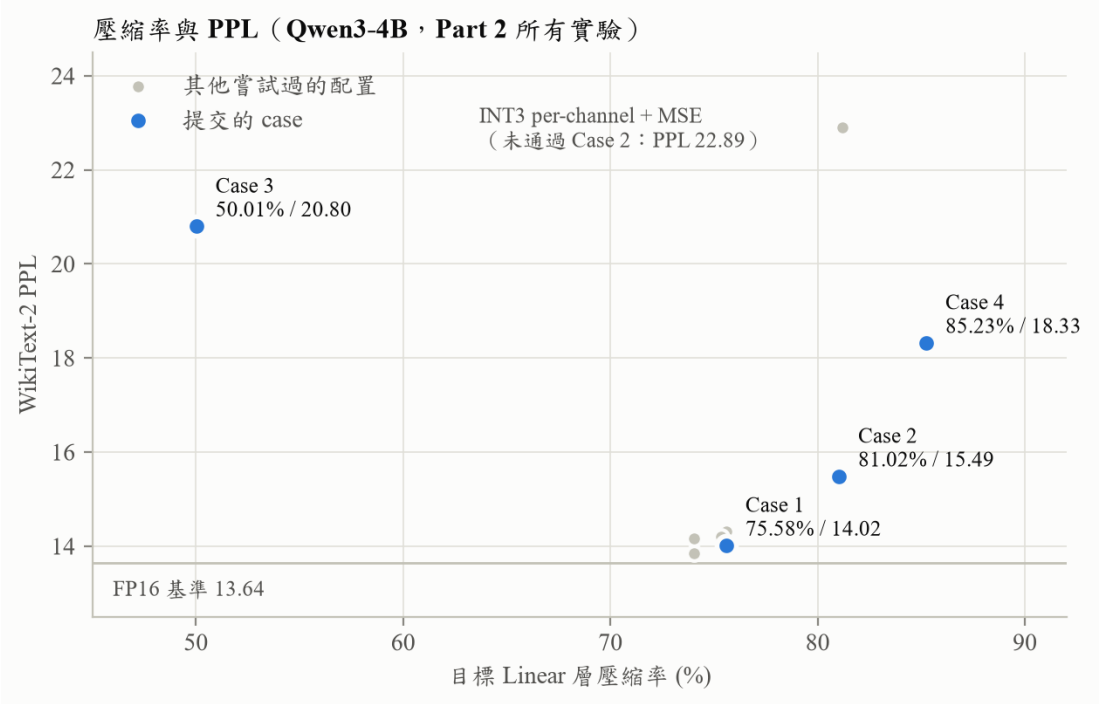


圖 4：壓縮率與 PPL（Part 2 所有實驗）

從 74% 到 81%，只靠量化時 PPL 只增加 0.2 到 1.9，代價很小。但到了 3-bit 以下就對 group 粒度非常敏感，per-channel 只比 g512 省 0.03 bits，PPL 卻從 15.49 變成 22.89。要省下同樣的空間，結構化剪枝（Case 4）的代價也比低秩分解（Case 3）小。

敏感層

以全部 INT4 g128 加 MSE（PPL 13.84）為基準，每次把一種 projection 降成 INT3 g128，結果如圖 5。gate_proj 和 down_proj 最不敏感，up_proj 和 q/o_proj 每個參數造成的 PPL 增加大約是它們的兩倍。

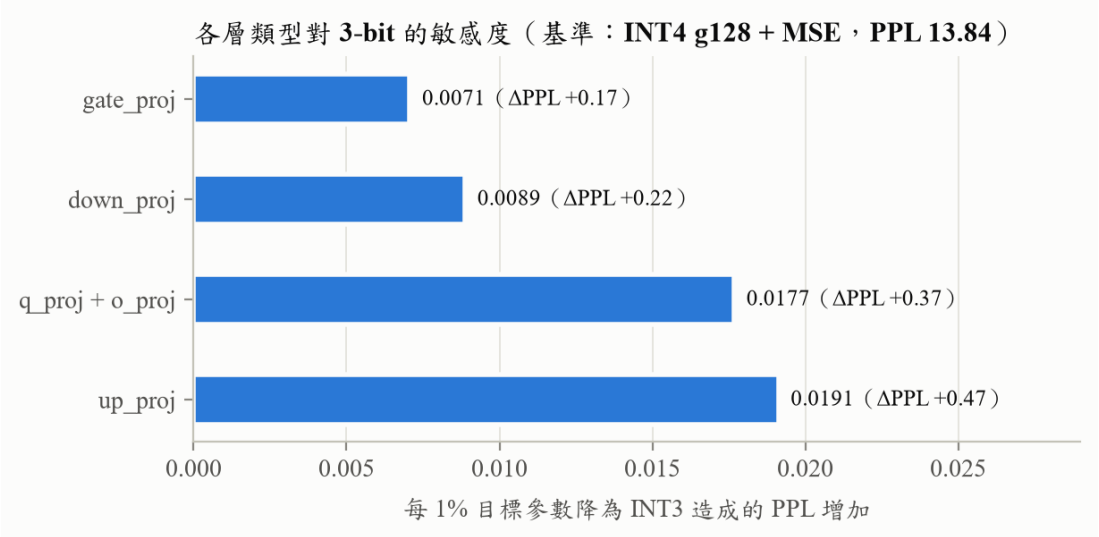


圖 5：各層類型對 3-bit 的敏感度

策略差異

Case 1、2 只靠量化，差別在混合精度和 group 大小。Case 3 受限於 SVD 至少 20% 的條件。Case 4 用結構化剪枝多換到約 4% 的壓縮率。

量化時有兩件事最有幫助。第一是 GPTQ，它的輸出誤差大約只有直接 round 的四分之一。第二是 MSE clipping，在 3-bit per-channel 時能讓誤差減少 56%。

限制

這次用的是 fake quantization，沒有真正的低 bit kernel，所以推論不會變快。混合精度只做到「層的類型」，沒有逐層調整。校正資料和測試資料都來自 WikiText-2，換成其他資料時效果可能較差。

結論

我認為最有效的方法是 GPTQ 加 MSE clipping 的 group-wise 量化。壓縮率要求越高，越應該先調 group 大小和混合精度，其次才是結構化剪枝，低秩分解的代價最高。這次最大的收穫是學到要先把 scale 和 zero point 的成本算清楚：全 INT4 過不了 75%、INT3 g128 過不了 81%，都是因為這些額外的 bits。